ИНТЕРНЕТ-ТЕХНОЛОГИИ

npm

Менеджер пакетов экосистемы JavaScript

Поляков Вячеслав 5040102/60201

Содержание

1. Что такое npm и где используется код из него
2. Зачем нужен менеджер пакетов
3. Проект с npm: package.json и его поля
4. Добавление пакетов и подключение в коде
5. Команды, виды зависимостей и версии (semver)
6. package-lock.json, node_modules и как npm ставит пакеты
7. npm scripts: свои команды проекта
8. Демонстрация: веб-приложение со списком задач
9. Публикация своего пакета
10. Безопасность

Что такое npm

npm – менеджер пакетов для JavaScript: программа, которая скачивает в проект готовые библиотеки.

Пакет

Готовая библиотека: папка с кодом и описанием (имя, версия, что ей нужно для работы)

Реестр npmjs.com

Сайт-хранилище пакетов: их там более 3 миллионов, опубликовать свой может любой

Команда npm

Программа, которую запускают в терминале (окне для текстовых команд): скачивает, обновляет и удаляет пакеты

Устанавливается вместе с Node.js – средой, которая запускает JavaScript вне браузера. Создан в 2010 году Айзеком Шлютером, с 2020 года принадлежит GitHub.

Где выполняется код пакетов

На сервере, в Node.js

Программа запускается на компьютере или сервере командой node

В todo-app: **express**, веб-сервер

В браузере

Код пакета загружается вместе со страницей и работает у пользователя

В todo-app: **dayjs**, форматирование дат

Язык один, JavaScript. Различается только место запуска. Node.js нужен самому npm и серверной части, но не браузеру пользователя.

Зачем нужен менеджер пакетов

Без npm

- Скачивать библиотеки вручную
- Копировать файлы в проект
- Самому следить за версиями
- Искать зависимости самих библиотек
- Присылать коллегам архив с библиотеками

С npm

- Одна команда: npm install
- Зависимости подтягиваются сами
- Версии записаны в файлах проекта
- Проект одинаково собирается у всех
- В репозитории только свой код

Веб-приложение для практической части: todo-app

Небольшое веб-приложение “Список задач”: в браузере можно добавить задачу, отметить её выполненной и удалить. Все примеры дальше взяты из него.

Сервер

Node.js и пакет **express**: хранит задачи и отдаёт их по запросам

Страница

HTML и JavaScript в браузере, даты форматирует пакет **dayjs**

Тесты

Проверяют сервер: пакет **supertest** и команда **npm test**

Пакеты, которые встретятся ниже (express, dayjs, supertest), – это то, что npm ставит в этот проект.

package.json: паспорт todo-app

```
{
  "name": "todo-app",
  "type": "module",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "dayjs": "^1.11.23",
    "express": "^5.3.0"
  },
  "devDependencies": {
    "supertest": "^7.3.1"
  }
}
```

name – имя

type – ES-модули

scripts – команды

dependencies – для работы

devDependencies – для разработки

Создаётся: **npm init**

Другие поля package.json

Поле	Для чего	Пример
`main`	главный файл	`"main": "server.js"`
`type`	формат модулей	`"type": "module"`
`engines`	версии Node.js	`"engines": { "node": ">=18" }`
`private`	запрет публикации	`"private": true`
`files`	что войдёт в пакет	`"files": ["dist"]`
`bin`	пакет как команда	`"bin": { "mycli": "cli.js" }`

Как добавить пакет в проект

1. **Найти**: на npmjs.com смотрим описание, число загрузок и дату обновления
2. **Установить**: `npm install dayjs` (для инструментов разработки добавляют -D)
3. **Что изменилось**: строка в dependencies, точные версии в package-lock.json, файлы в node_modules
4. **Подключить**: `import dayjs from "dayjs"` (ES-модули, как в todo-app) или `require("dayjs")` (старый формат CommonJS)

```
$ npm install dayjs
"dependencies": { "dayjs": "^1.11.23" }   <- запись в package.json
```

Управление пакетами

Команда	Что делает
`npm install`	ставит всё, что перечислено в package.json
`npm install имя`	добавляет пакет в dependencies
`npm install -D имя`	добавляет пакет в devDependencies
`npm install имя@5`	ставит конкретную версию (мажорную ветку)
`npm uninstall имя`	удаляет пакет и запись о нём
`npm ls`	показывает установленные пакеты
`npm outdated`	показывает пакеты, для которых вышли новые версии
`npm update`	обновляет в границах, разрешённых semver

Флаг **-g** ставит пакет глобально, как программу на компьютер, и в package.json проекта не записывается. Для библиотек проекта его не используют.

Два вида зависимостей

dependencies

Нужны приложению для работы

```
npm install express
```

В todo-app: **express**, **dayjs**

devDependencies

Нужны только при разработке

```
npm install -D supertest
```

В todo-app: **supertest**

```
$ npm install --omit=dev   # без dev-зависимостей
```

Версии: semver

`MAJOR.MINOR.PATCH  =  1.11.23`

- **MAJOR** – несовместимые изменения
- **MINOR** – новые возможности, старый код работает
- **PATCH** – исправления ошибок

Запись	Разрешённые версии
`^1.11.23`	от 1.11.23 до 2.0.0 (не включая): новые minor и patch
`~1.11.23`	от 1.11.23 до 1.12.0 (не включая): только patch
`1.11.23`	только эта версия

npm update остаётся в этих рамках. Перейти на новую мажорную версию можно вручную: **npm install имя@latest**.

package-lock.json и node_modules

package-lock.json

- Точные версии всех пакетов, включая вложенные
- Делает установку воспроизводимой
- Хранится в git

node_modules

- Папка со скачанными пакетами
- В todo-app: 3 прямые зависимости, 86 пакета в папке
- В git не хранится (.gitignore)

Команда	Как ставит
`npm install`	по package.json и lock-файлу; lock может обновиться
`npm ci`	стирает node_modules и заново ставит всё по lock-файлу; для сервера сборки и CI

Дерево зависимостей

```
todo-app
+-- dayjs
+-- express
|   +-- body-parser
|   |   `-- debug  (повтор)
|   `-- ... ещё 26
`-- supertest  (dev)
    `-- superagent
        `-- form-data
```

- 3 пакета, у каждого свои зависимости: **дерево**
- **debug** нужен многим, но ставится один раз
- Итого: 3 прямые зависимости, **86 пакетов**

`npm ls --all`

Что делает npm install

1. **Читает package.json**: нужные пакеты и диапазоны версий
2. **Смотрит package-lock.json**: есть – берёт точные версии, нет – подбирает по semver
3. **Спрашивает реестр**: какие версии есть и какие у них зависимости
4. **Строит дерево целиком**: убирает повторы (дедупликация)
5. **Скачивает архивы .tgz**: проверяет контрольную сумму, кеширует в ~/.npm
6. **Распаковывает в node_modules**: и создаёт или обновляет package-lock.json
7. **Запускает скрипты установки**: postinstall, prepare, если они есть

```
$ npm install
added 86 packages, and audited 87 packages
```

npm scripts: команды todo-app

```
"scripts": {
  "start": "node server.js",
  "dev": "node --watch server.js",
  "test": "node --test",
  "audit:check": "npm audit"
}
```

Команда	Что запустит
`npm start`	node server.js: запуск сервера
`npm run dev`	перезапуск сервера при изменении файлов
`npm test`	node --test: тесты API
`npm run audit:check`	npm audit: проверка уязвимостей

- Раздел **scripts** задаёт короткие имена для длинных команд
- **start** и **test** запускаются без run, остальные: **npm run имя**
- **npm run** без имени покажет список всех скриптов проекта

Популярные скрипты в проектах

Скрипт	Что делает	Типичная команда
start	запускает приложение	node server.js
dev	режим разработки	vite, next dev, node --watch
build	собирает проект для публикации	vite build, webpack, tsc
test	запускает тесты	jest, vitest, node --test
lint	ищет ошибки и нарушения стиля	eslint .

Имена выбирает автор проекта. Кроме **start** и **test** запуск такой: **npm run имя**.

Что ещё умеют скрипты

Программы из пакетов

Установили пакет eslint в проект – в скриптах его можно запускать по имени, глобально ставить не нужно

```
"lint": "eslint ."
```

Хуки pre и post

pretest запускается сразу перед test, а при ошибке test не стартует; posttest сразу после

```
"pretest": "npm run lint"
```

Аргументы

Всё после **--** уходит самой команде

```
npm test -- --watch
```

Цепочки

Скрипты соединяют через &&

```
"ci": "npm run lint && npm test"
```

Демо: веб-приложение «Список задач»

Браузер

- index.html: страница
- main.js: код страницы
- dayjs: формат даты

HTTP + JSON
`/api/tasks`
↔

Сервер (Node.js)

- server.js: запуск
- app.js: маршруты API
- express: сервер

Пакет	Для чего
`express (dependencies)`	веб-сервер и маршруты API
`dayjs (dependencies)`	формат даты в браузере; файл отдаётся из node_modules
`supertest (devDependencies)`	проверка API в тестах (npm test)

Демонстрация

```
$ cd src
$ npm install              # 3 пакета -> 86 в node_modules
$ npm test                 # 3 теста API
$ npm run dev              # http://localhost:3000
$ npm install uuid         # package.json: новая строка
$ npm uninstall uuid       # запись исчезла
$ npm audit                # found 0 vulnerabilities
```

Смотрим: package.json и package-lock.json до и после, папка node_modules, страница в браузере, вызов скрипта через npm run.

Свой пакет: публикация в npm

Как опубликовать

```
$ npm login
$ npm version patch
$ npm publish
```

- В package.json нужны name и version, часто main и files
- Имя должно быть уникальным в реестре
- Можно с областью: @автор/пакет

Зачем

- Делиться библиотекой с другими
- Общий код нескольких проектов команды
- Частные пакеты и корпоративные реестры (адрес реестра задаётся в файле .npmrc)
- Версия растёт по semver: npm version

Безопасность: случаи и защита

Известные случаи

- 2016, left-pad: автор удалил пакет, сломались сборки
- 2018, event-stream: вредоносный код после передачи пакета
- 2025, chalk и debug: подмена пакетов, червь Shai-Hulud

Как защищаться

- **npm audit**, **npm audit fix**
- Lock-файл в git, на сервере **npm ci**
- **--ignore-scripts** для чужих скриптов
- Dependabot, Renovate

Итоги

- npm – реестр, консольная утилита и часть экосистемы JavaScript: сервер, браузер, инструменты, десктоп и мобильные
- package.json описывает проект, lock-файл фиксирует версии, node_modules не хранят в git
- Пакеты добавляют командой npm install; -D для инструментов разработки; подключают через import (ES-модули) или require (CommonJS)
- Версии задаются по semver: ^ и ~ ограничивают обновления
- npm scripts дают проекту единые команды: start, test, dev, и др.
- npm ставит дерево зависимостей: из 3 прямых получилось 86 пакета
- Чужой код нужно проверять: npm audit, lock-файл, npm ci

Спасибо за внимание

Вопросы?